

Infosys SP/DSE - OOP Interview Notes

10 canonical interview answers for spaced repetition. Use these as the stable reference answers; expand naturally during an interview.

OOP Quick Recall Map

Topic	Recall anchor
OOP	Objects -> state + behavior
4 pillars	Encapsulation Inheritance Polymorphism Abstraction
Overloading	Same name + different parameters
Overriding	Child changes inherited behavior
Dynamic dispatch	Runtime method selection
Relationships	is-a -> inheritance has-a -> composition
Interface	Contract
Abstract class	Contract + shared implementation/state
Private fields	Controlled access to internal state

1. What is OOP?

Definition

Object-Oriented Programming (OOP) is a programming paradigm that organizes software around objects, which combine data (state) and methods (behavior) that operate on that data.

Why it exists / Why it matters

- OOP helps model real-world entities.
- It supports modularity, reusability, maintainability, and easier extension.

Example

A BankAccount object can contain state such as balance, account number, and owner, together with behavior such as deposit(), withdraw(), and check_balance().

Trade-off / Important note

OOP can introduce additional abstraction and complexity compared with simple procedural code, especially for small programs.

Spaced-repetition recall: Objects -> state + behavior -> modularity -> reusability -> maintainability

2. Explain the four pillars of OOP.

Definition

The four commonly taught pillars are Encapsulation, Inheritance, Polymorphism, and Abstraction.

Why it exists / Why it matters

- Encapsulation: bundle data and the methods that operate on it while controlling access to internal state.
- Inheritance: a child class derives properties and behavior from a parent class.
- Polymorphism: the same interface or method call can produce different behavior depending on the object.
- Abstraction: expose essential functionality while hiding unnecessary implementation details.

Example

Animal is a base abstraction; Dog and Cat can provide different speak() implementations.

Trade-off / Important note

The four pillars solve different design problems; using them excessively can make a simple system harder to understand.

Spaced-repetition recall: Encapsulation -> control data | Inheritance -> reuse/extend | Polymorphism -> different behavior | Abstraction -> hide complexity

3. Explain polymorphism with an example.

Definition

Polymorphism means that the same interface or operation can have different implementations or behavior depending on the object.

Why it exists / Why it matters

- It allows code to work with different object types through a common interface.
- Each object can provide its own implementation of the operation.

Code example

```
class Dog:
    def speak(self):
        print("Bark")

class Cat:
    def speak(self):
        print("Meow")

animals = [Dog(), Cat()]

for animal in animals:
    animal.speak()
```

The same call `animal.speak()` produces different behavior: Dog -> Bark, Cat -> Meow.

Trade-off / Important note

Polymorphism improves flexibility and extensibility, but excessive abstraction can make behavior harder to trace.

Spaced-repetition recall: Same interface -> different object -> different behavior

4. Overloading vs overriding

Key comparison

Aspect	Overloading	Overriding
Meaning	Same method name with different parameter lists	Child provides a new implementation of an inherited method
Relationship	Usually within the same class	Parent-child relationship
Parameters	Different	Typically the same method signature
Purpose	Multiple ways to call an operation	Specialize inherited behavior
Common association	Compile-time polymorphism	Runtime polymorphism

Code example

```
# Overloading in Java/C++ style
add(int a, int b)
add(int a, int b, int c)
```

```
class Animal:
    def speak(self):
        print("Some sound")

class Dog(Animal):
    def speak(self):
        print("Bark")
```

Important notes

- Python does not support traditional compile-time method overloading like Java/C++.
- Python commonly uses default arguments, `*args`, or `**kwargs` for similar flexibility.

- Dog overrides Animal.speak().

Spaced-repetition recall: Overloading -> same name + different parameters | Overriding -> child changes inherited behavior

5. Abstraction vs encapsulation

Key comparison

Concept	Core idea	Question it answers
Abstraction	Hide unnecessary implementation details and expose essential functionality	What should the user need to know?
Encapsulation	Bundle state + behavior and control access to internal state	How should internal state be protected/managed?

Example

For a car, a driver uses `car.start()` without needing to know every internal engine detail. For a bank account, controlled operations such as `deposit()` and `withdraw()` manage access to balance.

Spaced-repetition recall: Abstraction = WHAT to expose | Encapsulation = HOW to protect/manage

6. Interface vs abstract class

Definition

An interface primarily defines a contract that implementing classes agree to follow. An abstract class is a base class that can define a common abstraction and may contain both abstract behavior and concrete/shared behavior.

Key comparison

Aspect	Interface	Abstract class
Main purpose	Define a contract	Define a common base abstraction
Shared implementation	Usually limited / language-dependent	Can contain concrete methods
State	Usually not the main purpose	Can maintain object state
Relationship	Must implement this contract	Is a specialized form of this base

Example

A Payment interface can define `pay()`. `CreditCardPayment`, `UPIPayment`, and `CashPayment` can implement that contract. An abstract class could provide shared payment logic plus abstract methods.

Spaced-repetition recall: Interface -> contract | Abstract class -> contract + shared implementation/state

7. Composition vs inheritance

Key comparison

Aspect	Inheritance	Composition
Relationship	is-a	has-a
Meaning	Derive one class from another	Build an object using other objects
Example	Dog is an Animal	Car has an Engine
Typical benefit	Reuse/extension through a hierarchy	Flexibility and looser coupling

Code example

```
class Animal:
    pass

class Dog(Animal):
    pass

class Car:
    def __init__(self):
        self.engine = Engine()
```

Trade-off / Important note

Composition is often preferred when components should be replaceable without changing an inheritance hierarchy. Inheritance is appropriate when there is a genuine is-a relationship and polymorphic substitution makes sense.

Spaced-repetition recall: is-a -> inheritance | has-a -> composition

8. What is dynamic dispatch?

Definition

Dynamic dispatch is the mechanism where the implementation of an overridden method is selected at runtime based on the actual type of the object.

Why it exists / Why it matters

It enables runtime polymorphism.

Code example

```
class Animal:
    def speak(self):
        print("Animal sound")

class Dog(Animal):
    def speak(self):
        print("Bark")

animal = Dog()
animal.speak()
```

The runtime object is a Dog, so the call resolves to Dog.speak().

Spaced-repetition recall: Runtime object -> method lookup -> overridden implementation

9. Why use private fields?

Definition

Private or restricted fields are used to prevent uncontrolled direct access to an object's internal state and encourage access through controlled methods or interfaces.

Why it exists / Why it matters

- They support encapsulation.
- They help preserve data integrity and object invariants.
- They allow controlled modification and improve maintainability.

Example

Instead of allowing `account.balance = -50000`, the class can expose controlled operations such as `withdraw()` and `deposit()` that validate changes.

Python-specific note

Python does not enforce private fields in the same strict way as languages such as Java. Common conventions include `_balance` and name mangling with `__balance`.

Spaced-repetition recall: Private field -> protect internal state -> controlled access -> preserve invariants

10. Give a real-world example of polymorphism.

Definition

A payment system is a practical example: different payment methods expose the same operation, such as `pay()`, but implement it differently.

Why it exists / Why it matters

This lets the application add new payment implementations without rewriting code that only depends on the common interface.

Example

Credit Card, UPI, and PayPal can all provide `pay()`. The application can call `payment.pay()` without needing to know the exact payment type.

Spaced-repetition recall: One interface -> many implementations -> different behavior

Final Spaced-Repetition Recall

For reviews, look at the question title only and try to reproduce the canonical answer from memory before checking the note.

- Review 1: same day / end of study session
- Review 2: next day
- Review 3: after 3 days
- Review 4: after 7 days
- Review 5: after 14 days

Spaced-repetition recall: Memorize the structure and technical wording; explain naturally instead of trying to reproduce every sentence exactly.

10-Question Rapid Recall Drill

Cover the answers. Read only the question and answer aloud from memory.

- What is OOP?
- What are the four pillars of OOP?
- What is polymorphism?
- Overloading vs overriding?
- Abstraction vs encapsulation?
- Interface vs abstract class?
- Composition vs inheritance?
- What is dynamic dispatch?
- Why use private fields?
- Give a real-world example of polymorphism.